key difference between blockchain programming and a traditional distributed application development. How do you implement these rules and requirements?

Solidity provides various language features and functions that address these trust requirements. These language features are as follows:

- *Modifiers* specify access control rules to *verify* and manage who has control of data and functions to establish trust and privacy. (Perhaps only the chairperson can register members, for example.) These modifiers are also called *access modifiers* to distinguish them from the visibility modifiers (public and private) of functions and data.
- The `require(condition)` declaration *validates* the condition passed as a parameter and reverts the transaction if the check fails. This feature is commonly used for general validation of parameters (such as `age > 21`).
- The `revert()` statement allows you to *revert* a transaction and also prevent it from being recorded on the blockchain. This feature is commonly used in modifier definitions.
- The `assert(condition)` declaration *validate*s the condition of the variable or data during the execution of a function and reverts the transaction if the check fails. This feature is used for exceptions when you don't want the condition to fail, such as to validate the head count in the middle of the ocean during a cruise! Another example is to stop your bill payment if there is not enough money in your bank account.

3.2.7 Defining and using modifiers